

RETO TÉCNICO · QUALITY ENGINEERING

Automatización de Pruebas de APIs

Puesto	QA / QE Semi Senior
Dominio técnico	Automatización de pruebas de APIs REST
Tiempo estimado	1 hora 30 minutos
Modalidad de entrega	Repositorio público de GitHub + documentación
Stack	Libre elección (justificada por el candidato)

1. Propósito del reto

Evaluar la capacidad técnica, el criterio de calidad y el enfoque de ingeniería del candidato para diseñar, implementar y evidenciar una solución de automatización de pruebas de APIs, aplicando prácticas de Quality Engineering alineadas a mantenibilidad, trazabilidad, confiabilidad y mejora continua.

2. Contexto del negocio

Eres el QA responsable de garantizar la calidad de las APIs REST de una plataforma de e-commerce. Tu misión es diseñar e implementar un framework de automatización de pruebas de API que garantice la calidad y confiabilidad del sistema, validando las funcionalidades críticas de la plataforma de comercio electrónico.

3. Objetivos de evaluación

Este reto evalúa tu capacidad para:

- Diseñar una arquitectura de testing escalable y mantenible.
- Implementar validaciones funcionales y no funcionales.
- Manejar datos de prueba estáticos y dinámicos.
- Gestionar escenarios positivos, negativos y edge cases.
- Generar reportes y evidencias profesionales.
- Aplicar clean code y principios SOLID.

4. Recursos proporcionados

API a automatizar

API	Fake Store API
URL base	https://fakestoreapi.com
Documentación	https://fakestoreapi.com/docs
Endpoints	Products, Carts, Users, Auth

5. Requisitos funcionales

Debes implementar al menos 4 casos de prueba, combinando escenarios positivos y negativos según el siguiente detalle:

Escenarios positivos (selecciona 2 o más)

- **Caso 1 – Obtener producto específico**
 - GET a /products/{id}.
 - Validar status code 200.
 - Validar datos del producto (id, title, price, description, category, image, rating).
 - Validar tipos de datos correctos (price debe ser number).
 - Validar la estructura del objeto rating (rate, count).
 - Validaciones adicionales que consideres convenientes (opcional).
- **Caso 2 – Listar productos por categoría**
 - GET a /products/category/electronics.
 - Validar status code 200.
 - Validar que todos los productos retornados pertenecen a la categoría solicitada.
 - Validar la estructura de cada producto.
 - Validaciones adicionales que consideres convenientes (opcional).
- **Caso 3 – Crear producto exitosamente**
 - POST a /products con datos válidos.
 - Validar status code 200.
 - Validar que retorna el producto creado con ID.
 - Validar que los datos enviados coinciden con los retornados.
 - Validaciones adicionales que consideres convenientes (opcional).
- **Caso 4 – Actualizar producto completo**
 - PUT a /products/{id} con datos completos.
 - Validar status code 200.
 - Validar que los datos se actualizaron correctamente.
 - Validar todos los campos del producto.
 - Validaciones adicionales que consideres convenientes (opcional).

Escenarios negativos (selecciona 2 o más)

- **Caso 5 – Producto no encontrado**
 - GET a /products/9999999.
 - Validar status code.
 - Validar la respuesta de error.
 - Validaciones adicionales que consideres convenientes (opcional).
- **Caso 6 – Categoría inválida**
 - GET a /products/category/categoria-inexistente.
 - Validar status code.
 - Validar el manejo de error apropiado.
 - Validar que retorna un array vacío o un mensaje de error.
 - Validaciones adicionales que consideres convenientes (opcional).

- **Caso 7 – Crear producto con datos inválidos**
 - POST a /products con payload vacío o campos faltantes.
 - Validar status code.
 - Validar el manejo de errores apropiado.
 - Validar la estructura de la respuesta de error.
 - Validaciones adicionales que consideres convenientes (opcional).
- **Caso 8 – Validación de límites de productos**
 - GET a /products?limit=5.
 - Validar status code.
 - Validar que retorna un máximo de 5 productos.
 - Validar que el parámetro de paginación funciona correctamente.
 - Validaciones adicionales que consideres convenientes (opcional).

6. Stack tecnológico

Eres completamente libre de elegir el stack tecnológico con el que te sientas más cómodo. No hay restricciones de lenguaje, framework o herramientas. Lo importante es que demuestres:

- Dominio de las herramientas que elijas.
- Justificación de tu elección.
- Implementación profesional y de calidad.
- Capacidad para automatizar APIs de forma robusta.

7. Entregables

7.1 Código fuente

- Repositorio Git con el proyecto de automatización.

7.2 Reporte de ejecución

Puede ser el reporte propio de la herramienta o uno personalizado, e incluir:

- Total de pruebas ejecutadas.
- Tasa de éxito / fallo.
- Tiempo de ejecución por prueba.
- Detalle de fallos con request/response completos:
 - Request completo (URL, method, headers, body).
 - Response completo (status, headers, body).
 - Resultado de las validaciones.

7.3 Documentación

- Archivo README.md con instrucciones claras de ejecución, tecnologías y versiones utilizadas, de modo que la implementación pueda replicarse.
- Archivo EVALUATION.md (u otros) con conclusiones y hallazgos del ejercicio.
- En caso de usar capacidades de IA, especificar los MCP, Skills y Agentes utilizados.

8. Criterios de evaluación

La solución será valorada de forma integral considerando los siguientes criterios:

Criterio	Qué se evalúa	Peso
Arquitectura y diseño	Estructura del framework, escalabilidad, mantenibilidad y aplicación de SOLID/clean code.	25%
Cobertura de casos	Selección y correcta implementación de escenarios positivos, negativos y edge cases.	25%
Calidad de validaciones	Robustez y pertinencia de las aserciones (status, esquema, tipos de dato).	20%
Reportería y evidencias	Claridad, completitud y profesionalismo del reporte de ejecución.	15%
Documentación y reproducibilidad	README claro, versiones especificadas y facilidad de replicación.	15%

Nota: el uso de buenas prácticas de ingeniería y la claridad en la justificación de las decisiones técnicas serán valoradas positivamente por el panel evaluador.